

Git 常用命令指南

本文整理 Git 日常开发中高频使用的命令，按功能模块分类，便于快速查阅和实操。

一、基础配置（首次使用必做）

用于设置本地 Git 的用户信息、默认编辑器等，配置信息保存在~/.gitconfig（全局）或项目.git/config（局部）中。

命令	作用	示例
git config --global user.name "用户名"	配置全局用户名（关联提交记录）	git config --global user.name "ZhangSan"
git config --global user.email "邮箱"	配置全局邮箱（需与远程仓库账号匹配）	git config --global user.email "zhangsan@xxx.com"
git config --global core.editor "编辑器"	配置默认提交编辑器（如 VS Code）	git config --global core.editor "code --wait"
git config --list	查看当前 Git 配置（全局 + 局部）	git config --list
git config --global --unset 配置项	删除全局配置项	git config --global --unset user.email

二、仓库操作（初始化 / 克隆 / 查看）

用于创建本地仓库、克隆远程仓库，或查看仓库状态。

命令	作用	示例
git init	在当前目录初始化本地 Git 仓库（生成.git文件夹）	进入项目文件夹后执行：git init
git clone <远程仓库地址>	克隆远程仓库到本地（生成与仓库同名的文件夹）	git clone https://github.com/xxx/xxx.git

git clone <远程地址> <自定义文件夹名>

克隆远程仓库并指定本地文件夹名	<code>git clone https://github.com/xxx/xxx.git my-project</code>	
<code>git status</code>	查看工作区、暂存区的文件状态（已修改 / 未跟踪等）	<code>git status</code> （常用，建议每次提交前执行）

三、分支管理（核心功能）

Git 的核心优势之一，用于创建分支、切换分支、合并分支等，避免开发冲突。

命令	作用	示例
<code>git branch</code>	查看本地所有分支（当前分支前标*）	<code>git branch</code>
<code>git branch -r</code>	查看远程所有分支	<code>git branch -r</code>
<code>git branch -a</code>	查看本地 + 远程所有分支	<code>git branch -a</code>
<code>git branch <分支名></code>	创建新分支（基于当前分支）	<code>git branch feature/login</code> （创建“登录功能”分支）
<code>git checkout <分支名></code>	切换到指定分支	<code>git checkout feature/login</code>
<code>git switch <分支名></code>	（Git 2.23 + 新增）切换分支（更直观，替代checkout）	<code>git switch feature/login</code>
<code>git checkout -b <分支名></code>	创建并立即切换到新分支	<code>git checkout -b feature/pay</code> （一步完成创建 + 切换）
<code>git switch -c <分支名></code>	（Git 2.23+）创建并切换新分支（同上面命令）	<code>git switch -c feature/pay</code>
<code>git merge <待合并分支名></code>	将“待合并分支”合并到 当前分支	切换到main分支后： <code>git merge feature/login</code> （合并登录功能到主分支）
<code>git branch -d <分支名></code>	删除本地已合并的分支（未合并会提示）	<code>git branch -d feature/login</code>

<code>git branch -D <分支名></code>	强制删除本地分支（无论是否合并，谨慎使用）	<code>git branch -D feature/old</code>
<code>git push origin --delete <远程分支名></code>	删除远程分支	<code>git push origin --delete feature/login</code>

四、提交操作（工作区→暂存区→本地仓库）

Git 的“三阶段”提交流程：工作区修改→暂存区暂存→本地仓库提交。

命令	作用	示例
<code>git add <文件名></code>	将单个文件从“工作区”添加到“暂存区”	<code>git add src/login.js</code>
<code>git add .</code>	将当前目录下 所有已修改 / 新增 的文件添加到暂存区（不包含删除文件）	<code>git add .</code> （常用，提交前暂存所有变更）
<code>git add -A / git add --all</code>	将当前目录下 所有变更 （修改 / 新增 / 删除）添加到暂存区	<code>git add -A</code>
<code>git commit -m "提交备注"</code>	将“暂存区”的文件提交到“本地仓库”，备注需清晰描述变更（如“修复登录验证码错误”）	<code>git commit -m "fix: 登录验证码过期问题"</code>
<code>git commit --amend</code>	修改 最近一次 的提交备注（若暂存区有新文件，会一并加入上次提交）	执行后进入编辑器修改备注，保存退出即可
<code>git log</code>	查看本地仓库的提交历史（按时间倒序，包含作者、时间、备注、commit ID）	<code>git log</code>
<code>git log --oneline</code>	简洁查看提交历史（每行显示：简短 commit ID + 提交备注）	<code>git log --oneline</code> （常用，快速定位历史）

五、远程仓库交互（拉取 / 推送 / 关联）

与 GitHub、GitLab 等远程仓库同步代码。

命令	作用	示例
<code>git remote</code>	查看已关联的远程仓库名称（默认通常是origin）	<code>git remote</code>
<code>git remote -v</code>	查看远程仓库的详细地址（fetch 拉取地址、push 推送地址）	<code>git remote -v</code>
<code>git remote add <远程名> <远程地址></code>	关联远程仓库（首次推送本地仓库到远程时使用）	<code>git remote add origin https://github.com/xxx/xxx.git</code>
<code>git pull <远程名> <远程分支名></code>	拉取远程分支的代码到 当前本地分支 （自动合并，若有冲突需先解决）	<code>git pull origin main</code> （拉取远程 main 分支到本地当前分支）
<code>git push <远程名> <本地分支名>:<远程分支名></code>	推送本地分支代码到远程分支（若远程分支不存在则自动创建）	<code>git push origin feature/login:feature/login</code>
<code>git push <远程名> <本地分支名></code>	简化写法：若本地分支与远程分支同名，可省略远程分支名	<code>git push origin feature/login</code>
<code>git push -u <远程名> <本地分支名></code>	首次推送分支时，绑定“本地分支”与“远程分支”（后续可直接用git push）	<code>git push -u origin feature/login</code> （绑定后，下次推送直接git push）

六、撤销与回退（救急必备）

用于撤销工作区 / 暂存区的变更，或回退到历史版本（谨慎操作！）。

命令	作用	示例	注意事项
<code>git checkout -- <文件名></code>	撤销工作区中该文件的 未暂存 修改（恢复到最近一次提交 / 暂存的状态）	<code>git checkout -- src/login.js</code>	仅对未add到暂存区的文件有效，已暂存的需先撤销暂存

<code>git reset HEAD <文件名></code>	将暂存区中的该文件“撤回到”工作区（取消暂存）	<code>git reset HEAD src/login.js</code>	撤销后文件回到“已修改但未暂存”状态
<code>git reset --hard <commit ID></code>	回退本地仓库到指定commit ID的版本（工作区、暂存区也会同步回退）	先通过 <code>git log -oneline</code> 获取 ID：- <code>git reset --hard a1b2c3d</code>	高危操作！ 会彻底删除指定版本后的所有提交，未提交的变更也会丢失
<code>git stash</code>	暂存当前工作区的未提交变更（用于临时切换分支，后续可恢复）	<code>git stash</code> （暂存后工作区恢复干净状态）	暂存的内容保存在“stash 栈”中
<code>git stash pop</code>	恢复“stash 栈”中最近一次暂存的内容到工作区，并删除该暂存记录	<code>git stash pop</code>	
<code>git stash list</code>	查看“stash 栈”中所有暂存记录	<code>git stash list</code>	

七、实用技巧

1. 忽略文件（.gitignore）

在项目根目录创建.gitignore文件，写入不需要 Git 跟踪的文件 / 文件夹（如node_modules/、dist /、.env），Git 会自动忽略这些内容。

示例.gitignore内容：

```
# 忽略node_modules文件夹
node_modules/
# 忽略dist文件夹
dist/
# 忽略.env文件
.env
# 忽略所有.log文件
*.log
```

2. 解决合并冲突

执行`git merge`或`git pull`时若出现冲突，Git 会在冲突文件中用<<<<<< HEAD（当前分支内容）、
=====（待合并分支内容）、>>>>>> 分支名标记冲突位置，需手动修改文件保留正确内容，然后执行`git add` 冲突文件→`git commit -m "解决合并冲突"`完成合并。

3. 查看分支关联关系

执行`git branch -vv`，可查看本地分支与远程分支的绑定关系（如`feature/login a1b2c3d [origin/feature/login]` 修复登录bug）。

（注：文档部分内容可能由 AI 生成）